Partial Observability

CISC 7404 - Decision Making

Steven Morad

University of Macau

Quiz 2
Admin 4
Are MDPs Enough? 6
Partial Observability 8
Coding 10
Multiagent RL 14
Summary 16

Quiz

Quiz

I reported some cheaters to the department for quiz 2

- Cheating will result in 0 for **entire course**

It is not worth risking a 0 in the course for a quiz

- 10% or less of your course grade

Consider the expected return for failing one exam or the course

$$\mathbb{E}[\mathcal{G} \mid s_0 = \text{No study}, a_0 = \text{Cheat}] \approx 40\%$$

$$\mathbb{E}[\mathcal{G} \mid s_0 = \text{No study}, a_0 = \text{No cheat}] \approx \text{current_grade} - 5\%$$

I cannot catch all cheaters, but is it worth the gamble?

Quiz

- After all students put away their computer/notes/phone I will hand out exams
- If you have computer/notes/phone out after this point, it counts as cheating
- I will hand out exams face down, do not turn them over until I say so
- After turning them over, I will briefly explain each question
- After my explanation, you will have 75 minutes to complete the exam
- After you are done, give me your exam and go relax outside, we resume class at 8:30

Quiz

- There may or may not be different versions of the exam
- If your exam has the answer for another version, it is cheating
- Instructions are in both english and chinese, english instructions take precedence
- Good luck!

Quiz

- 在所有学生收起电脑/笔记/手机后,我会分发试卷。
- 如果在此之后仍有电脑/笔记/手机未收,将视为作弊。
- 试卷会背面朝下发下,在我宣布开始前请勿翻面。
- 试卷翻面后,我会简要说明每道题的注意事项。
- 说明结束后,你们有 75 分钟完成考试。
- 交卷后请到教室外休息,8:30 恢复上课。
- 试卷可能存在不同版本,细节略有差异。
- 若你的试卷上出现其他版本的答案,将被判定为作弊。
- 试卷说明为中英双语,若内容冲突以英文为准。
- 祝各位考试顺利!

Admin

Admin

Admin

Still one quiz 2 without name

If you took quiz 2 and have 0 score, it may be your quiz

Visit my office hours next week to claim it

Admin

Homework 2 grades released

- Average score 80/100
- Multimodal, modes at 100, 80, 40
- Changing evaluation random seed until success, not ok
- Difficult assignment, because RL is difficult!
  - CartPole/LunarLander is MNIST equivalent for RL
  - Today, still lots of trial and error, tuning, etc
  - Maybe one day it will be easy like supervised learning

Admin

Current mean course grade is 88%

- Mean score will decrease after quiz 3
- Mean score will decrease after finalizing participation
- Hopefully, mean score will increase after final project

If mean course grade drops below 80%, I will normalize upward

Admin

Almost done with the course!

Does the course differ from your expectations?

If you understand the material, you are part of a very small group

- $< 5,000$ people in the world truly understand RL
  - Many use RL, few understand it
    - MDP, policy gradient mechanism, γ meaning, PPO vs DDPG, etc
    - To really succeed with RL, you must understand it (not like SL)
- You are part of this elite group!

Admin

With these skills, you can do what others cannot

You have the tools to make **better decisions** than most humans

Humans can be:

- Biased (Incorrect modeling of Tr)
- Irrational (Making decisions while ignoring $\mathcal{G}(\boldsymbol{\tau})$)
- Shortsighted (γ too small)
- Greedy ($\mathcal{R}(s)$ that does not consider other humans)
- Evil (Positive $\mathcal{R}(s)$ for hurting other humans)

I hope you can create agents that do not share our flaws

- Please use your skills to improve the world

Are MDPs Enough?

Are MDPs Enough?

Problems with well-defined losses and gradients can use supervised/ unsupervised learning

- Generating or classifying cat pictures

RL is the most practical method for optimization of **arbitrary** objectives

- Reduce world poverty

Fewer "rules" for RL problems than supervised learning

- Only rule: problem must be MDP

Today, we will make RL even more flexible

- Remove almost all rules

Are MDPs Enough?

Throughout the course, I talk about useful applications of RL

- Medicine
- Material science
- Robotics
- Economics

Yet many RL successes focus on video games or board games

- AlphaStar (StarCraft II)
- AlphaGo (Go)
- OpenAI Five (DoTA II)
- DQN (Atari, PyBoyEnv)
- PPO/SAC (MuJoCo)

Are MDPs Enough?

We could be solving real problems, but we are not

Question: Why does RL research focus on solving video games?

Answer?: RL researchers are nerd gamers and only care about games

Are MDPs Enough?

Question: Why does RL research focus on solving video games?

Real Answer:

- RL requires lots of data, video games can generate data for free
  - Robots/medicine/etc require physical work to collect each s, a, r, d
- Video games are more reproducible
  - Same sequence of actions results in same outcome
    - E.g., material science, labs perform actions differently
      - Different centrifuge, reagent purity, etc
      - Makes it hard to rely on other labs results

Are MDPs Enough?

There is one more reason, one I want to cover today

- Video games are almost always MDPs

Remember, RL can **only** solve MDPs

If it not an MDP, we cannot solve it with RL

Are MDPs Enough?

Recall the definition of MDP

Definition: A Markov decision process (MDP) is a tuple (S, A, T, R, γ)

- S is the state space
- A is the action space
- $\mathrm{Tr} : S \times A \mapsto \Delta S$ is the state transition function
- $\mathcal{R} : S \mapsto \mathbb{R}$ is the reward function
- $\gamma \in [0, 1]$ is the discount factor

Question: Why do we call it **Markov** decision process?

Answer: Markov state transition function

Are MDPs Enough?

An MDP must have a Markov state transition function

Question: Does anyone remember mathematical definition for this?

The next state only depends on the current state and action

$$\mathrm{Tr}(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, ..., s_0, a_0) = \mathrm{Tr}(s_{t+1} \mid s_t, a_t)$$

Another way of saying this:

- Everything our agent saw in the past is **completely useless**
- Provides zero value for the current decision

Question: Do you think this is a strong constraint?

Are MDPs Enough?

Example: Simple MDP with sensor noise

$$S = \{0, 1\}, A = \{a\}, \mathcal{R}(s) = 0$$

We sense a small amount of noise ε in each state

$$s = 0 + \varepsilon \qquad s = 1 + \varepsilon \qquad \varepsilon \sim \text{Normal}(0, 1)$$

$$\text{Tr}(s_2 \mid s_1, a_1) = \text{Tr}(s_2 \mid s_1, a_1, s_0, a_0)$$

$$\text{Tr}(s_2 = 1 \mid 0.91, a) \stackrel{?}{=} (s_2 = 1 \mid 0.91, a, 0.99, a)$$

Question: Do you think the probability distribution changes?

Answer: Yes, two states near 1 make it more likely we are in $s = 1$

Are MDPs Enough?

This is even more clear with high-noise samples

$$S = \{0, 1\}, A = \{a\}, \mathcal{R}(s) = 0$$

We sense a small amount of noise ε in each state

$$s = 0 + \varepsilon \qquad s = 1 + \varepsilon \qquad \varepsilon \sim \text{Normal}(0, 3)$$

$$\text{Tr}(s_2 \mid s_1, a_1) = \text{Tr}(s_2 \mid s_1, a_1, s_0, a_0)$$

$$\underbrace{\text{Tr}(s_2 = 1 \mid 0.68, a)}_{\text{Likely } s_2=1} \stackrel{?}{=} \underbrace{\text{Tr}(s_2 = 1 \mid 0.68, a, 0.01, a)}_{\text{Likely } s_2=0}$$

If we cannot determine the state, RL does not work

Are MDPs Enough?

Question: Can we have noise-free state in the real world?

Answer: No. Heisenberg uncertainty principle says noise-free measurement is impossible

Question: Noise-free state in virtual worlds (video games)?

Answer: Yes, Tr does not follow real world physics

Are MDPs Enough?

Wing Commander: shoot aliens

- Position of alien (cockpit/radar)
- Our heading and velocity
- Our shields and hull
- Target shields and hull

Question: Is there noise in these measurements (state)? **Answer:** No

Question: Do you think Wing Commander is an MDP? **Answer:** Yes

Are MDPs Enough?

Let us train an autonomous dentist

s_0: Patient arrives at dentist

a_0: **Dentist:** How often do you brush your teeth?

s_1: **Patient:** Once per week

a_1: **Dentist:** Ok, I must check for cavities, and possibly drill your teeth

s_2: **Patient:** Actually, I brush my teeth twice per day

$s_3 \in \{\text{Checking for cavities}, \text{Send patient home}\}$

Question: Is this an MDP? Will s_1 influence s_3? **Answer:** No

Are MDPs Enough?

The Markov property is very constraining

- Prevents applying RL to many real problems
- Instead, we focus on Markov video games

Problems with hidden information or noisy sensors are not MDPs

- No lie detector for humans
- Obscured cameras
- Sensor noise

Any problem where prior states contain useful information

- Playing two games against the same chess player
  - Chess player has specific behaviors

MDP traps RL in video games

Partial Observability

Partial Observability

So the MDP is not a realistic model for most problems

The core issue, is noise or ambiguity in the state

- Unrealistic that all prior states contain zero useful information

Can we formulate a new decision process that accounts for this noise/ambiguity?

The key idea is to separate **observations** from state

- Observations are what we observe/see
- The state is the hidden/unknown

We can use observations to infer/guess the state

Partial Observability

Definition: A **Partially Observable** MDP (POMDP) represents a Markov process where the state is hidden

$$((S, A, \mathrm{Tr}, \mathcal{R}, \gamma), \mathfrak{O}, O)$$

$\mathfrak{O}$: observation space

O: observation function

Cannot directly observe state $s \in S$, only observation $o \in \mathfrak{O}$

$$O : S \mapsto \Delta\mathfrak{O} \qquad o_{t+1} \sim O(\cdot \mid s_{t+1}) \qquad o_{t+1} \sim \Pr(\cdot \mid s_{t+1})$$

$$\mathrm{Tr}(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, ..., s_0, a_0) = \mathrm{Tr}(s_{t+1} \mid s_t, a_t)$$

$$\Pr(o_{t+1} \mid o_t, a_t, o_{t-1}, a_{t-1}, ..., o_0, a_0) \neq \Pr(o_{t+1} \mid o_t, a_t)$$

Partial Observability

With POMDPs, Markov Tr, S still governs the system

However, we do not know Tr or S, just $\mathfrak{O}$!

Example: Can model the stock market as a POMDP

- Complex and unknown S, Tr
  - S: Stock holdings of millions of people, banks, global events, etc
  - Tr: Cause and effect of banking systems, markets, emotions, etc
- Stock price is an observable function of latent state $s_t \in S$
  - $o_t \sim O(\cdot \mid s_t)$

POMDPs can model almost any type of decision making problem

Partial Observability

POMDPs seem very flexible, but how are they useful?

All RL/IL algorithms rely on MDPs, not POMDPs

Did you spend 12 weeks learning a pointless skill?

You are in luck, my research focus is on RL for POMDPs!

- *Deep Memory Models and Efficient Reinforcement Learning under Partial Observability*

So how can we apply RL/IL to POMDPs?

Partial Observability

The core issue is the Markov property of Tr

$$\mathrm{Tr}(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, ..., s_0, a_0) = \mathrm{Tr}(s_{t+1} \mid s_t, a_t)$$

However, in POMDPs, we receive observations o not states s

If prior observations $o_{t-1}, o_{t-2}, ...$ provide useful info, not Markov

Question: How to ensure that we have enough info to make decisions?

Answer: Just consider all observations $o_0, o_1, ...o_t$ at time t

Partial Observability

In fact, we can treat all prior observations and actions as a Markov state

$$s_t = \begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}$$

Partial Observability

$$\mathrm{Tr}(s_{t+1} \mid s_t, a_t, \ldots, s_0, a_0) = \mathrm{Tr}(s_{t+1} \mid s_t, a_t) \qquad s_t = \begin{bmatrix} o_t & a_t \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}$$

Plug s_t into Tr

$$\mathrm{Tr}\left(s_{t+1} \middle| \begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}, a_t, \begin{bmatrix} o_{t-1} & \emptyset \\ o_{t-2} & a_{t-2} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}, a_{t-1}, \ldots [o_0 \;\; \emptyset], a_0 \right)$$

Partial Observability

$$\mathrm{Tr}\left(s_{t+1} \middle| \begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}, a_t, \begin{bmatrix} o_{t-1} & \emptyset \\ o_{t-2} & a_{t-2} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}, a_{t-1}, \ldots \begin{bmatrix} o_0 & \emptyset \end{bmatrix}, a_0 \right)$$

$$= \mathrm{Tr}\left(s_{t+1} \middle| \begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}, a_t \right)$$

Question: Does this equality hold?

Answer: Yes, because s_t contains all other $s_{t-1}, s_{t-2}, \ldots$

Partial Observability

$$s_t = \begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}$$

With this state, POMDP becomes an MDP

Can solve this MDP using policy gradient, Q learning, etc

Still guaranteed to learn optimal policy $\arg\max_{\theta_\pi} \mathbb{E}[\mathcal{G}(\boldsymbol{\tau}) \mid s_0; \theta_\pi]$

Partial Observability

Note: Optimal policy is always defined by sensors

- Navigation policy for a “blind” robot may not be great
- But we guaranteed to learn best navigation policy for blind robot

Beneficial to select the most informative state/observation space

Partial Observability

$$s_t = \begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}$$

Question: Any problems with this state?

Answer: It can grow very large!

Example: $\mathfrak{O} = \{0, \ldots, 255\}^{1024 \times 768 \times 3}$

60 images per second over one hour, then state is 500 GB

Next hour, state becomes 1 TB – not scalable!

Partial Observability

$$s_t = \begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}$$

Must consider all prior observations and actions, but can we do better?

Key idea: Must consider all prior info, but not all prior info is important

- We can delete useless information
- We can summarize similar information

Partial Observability

$$s_t = \begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}$$

Use a function f to extract useful/meaningful information from the past

$$s_t = f\left(\begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}\right)$$

Partial Observability

$$s_t = f\left(\begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}\right)$$

Use a function f to extract useful/meaningful information from the past

Question: Is there a similar mechanism in humans?

Answer: Memory!

Partial Observability

$$s_t = f\left(\begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}\right)$$

Organisms receive observations over time

- See, smell, hear, ...

They summarize all sensory information into a compact representation

"Grab saw", "hold log", "move saw forward/backward on log"

Partial Observability

$$s_t = f\left(\begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}\right)$$

How can we implement f?

- Functions that take a dynamic number of inputs
- Many possibilities for f

Partial Observability

Approach 1: Recurrent neural networks

$$\mathrm{GRU}\big([o_t, a_{t-1}], h_{t-1}, \theta_f\big)$$

```
# Compute markov state s_t
recurrent_state = zeros()
a[i] = 0 # Most recent action is null
for i in range(len(trajectory)):
    input = concatenate(o[i], a[i])
    recurrent_state = gru(input, recurrent_state)
markov_state = recurrent_state
```

Partial Observability

Approach 2: Transformers/attention

$$s_t = \text{attn}\big([o_t, a_{t-1}], [o_{t-1}, a_{t-2}], \ldots, \theta_f\big)$$

```
# Compute markov state s_t
a[i] = 0 # Most recent action is null
input = concatenate(o[i], a[i])
markov_state = attn(input)
```

Partial Observability

Approach 3: Bayesian inference

$$\Pr\left(s_t \mid o_t, a_{t-1}, s_{t-1}, \ldots; \theta_f\right)$$

```
# Compute markov state s_t
mu, sigma = zeros(), ones() # Initialize parameters
a[i] = 0 # Most recent action is null

for i in range(len(trajectory)):
    mu, sigma = update_belief(mu, sigma, o[i], a[i])
state_dist = (mu, sigma)
# Either MLE or sample
markov_state = mu
markov_state = normal(mu, sigma)
```

Partial Observability

We train memory θ_f and policy θ_π end to end

$$s_t = f\big([o_t, a_{t-1}], [o_{t-1}, a_{t-2}], \ldots, \theta_f\big)$$

$$a_t \sim \pi(\cdot \mid s_t; \theta_\pi)$$

$$\arg\max_{\theta_f, \theta_\pi} \mathbb{E}[\mathcal{G}(\boldsymbol{\tau}) \mid s_0; \theta_\pi]$$

In this way, we **learn** the Markov state s_t using parameters θ_f

Example: Stock market POMDP

- Learn θ_f that represents Tr, S from data
- Learn θ_π to maximize profits using learned Tr, S

Partial Observability

Definition: We use a memory function f to apply standard RL algorithms (policy gradient, Q learning, actor critic) to POMDPs

$$s_t = f\big([o_t, a_{t-1}], [o_{t-1}, a_{t-2}], \ldots, \theta_f\big)$$

$$\arg\max_{\theta_f, \theta_\pi} \mathbb{E}[\mathcal{G}(\boldsymbol{\tau}) \mid s_0; \theta_\pi] = \arg\max_{\theta_f, \theta_\pi} \sum_{n=0}^{\infty} \gamma^n \sum_{s_{n+1} \in S} \mathcal{R}(s_{n+1}) \cdot \Pr(s_{n+1} \mid s_0; \theta_\pi)$$

$$\Pr(s_{n+1} \mid s_0; \theta_\pi) = \sum_{s_1, \ldots, s_n \in S} \prod_{t=0}^{n} \left(\sum_{a_t \in A} \mathrm{Tr}(s_{t+1} \mid s_t, a_t) \cdot \pi(a_t \mid s_t; \theta_\pi) \right)$$

Partial Observability

$$s_t = f\big([o_t, a_{t-1}], [o_{t-1}, a_{t-2}], \ldots, \theta_f\big)$$

$$\arg\max_{\theta_f, \theta_\pi} \mathbb{E}[\mathcal{G}(\boldsymbol{\tau}) \mid s_0; \theta_\pi] = \arg\max_{\theta_f, \theta_\pi} \sum_{n=0}^{\infty} \gamma^n \sum_{s_{n+1} \in S} \mathcal{R}(s_{n+1}) \cdot \Pr(s_{n+1} \mid s_0; \theta_\pi)$$

$$\Pr(s_{n+1} \mid s_0; \theta_\pi) = \sum_{s_1, \ldots, s_n \in S} \prod_{t=0}^{n} \left(\sum_{a_t \in A} \mathrm{Tr}(s_{t+1} \mid s_t, a_t) \cdot \pi(a_t \mid s_t; \theta_\pi) \right)$$

Difficult optimization problem

- Difficulty increase combinatorially with trajectory length
- Deep models are surprisingly capable and generalize well
  - In practice, do not need combinatorially more samples
  - Still more expensive than RL on MDPs

Partial Observability

POMDPs are the most flexible decision making process I know

I have yet to see a problem that POMDPs cannot represent

We can solve POMDPs by combinining memory and standard RL

Today, we have enough compute to solve simpler tasks

- Visual navigation
- Board games
- IQ memory exams

As GPUs become faster, we will see very interesting results

- Your generation will see RL applied to useful tasks
- You can be the ones to apply it!

Coding

Coding

```
f = Transformer() # or RNN
pi = nn.Sequential(..)
episode = []
while not terminated:
    a = sample_action(f, pi, episode["o"], episode["a"])
    episode.append(env.step(action))
    next_state = f(episode['o'])
```

Coding

When sampling actions, we must compute f over the full trajectory

```
def sample_action(f, pi, obs, action):
    # trajectory up to timestep t
    traj = concatenate([obs, action])
    state = f(traj) # s_t
    a = pi(state)
```

Coding

We can utilize existing loss functions

Implement a wrapper to compute states then call regular loss

```
def recurrent_loss(models, episode, loss_fn=pg_loss):
    f, pi = models
    # You can parallelize this to improve speed
    # I write it out for clarity
    states = []
    for t in range(len(episode)):
        states.append(f(episode["o"], episode["a"]))
    states = array(states)
    return loss_fn(pi, states, episode)
```

Multiagent RL

Multiagent RL

We model multiagent RL as a POMDP with additional structure

In a Decentralized MDP (DecMDP), the state is distributed across agents

Consider observations over agents instead of time

Temporal POMDP

$$s_t = \begin{bmatrix} o_t & \emptyset \\ o_{t-1} & a_{t-1} \\ \vdots & \vdots \\ o_0 & a_0 \end{bmatrix}$$

DecMDP

$$s_t = \begin{bmatrix} o_{n,t} \\ o_{n-1,t} \\ \vdots \\ o_{0,t} \end{bmatrix}$$

Multiagent RL

What methods can we use for multiagent f?

- Attention/transformers
- Graph neural networks
- RNNs not a good idea, not permutation invariant
  - Time has ordering, agents do not

Multiagent RL

Centralized Training Decentralized Execution (CTDE) is a popular approach for multiagent RL

- Multiagent DDPG (MADDPG)
- Multiagent SAC (MASAC)

Key idea: Non-symmetric Q and π

- Learn Q function over joint action space
  - $Q : \mathfrak{O}_0 \times \mathfrak{O}_1 \times \ldots \times \mathfrak{O}_n \mapsto A_0 \times A_1 \times \ldots \times A_n$
  - Expensive to compute, only necessary during training
- Learn independent policies $\pi : \mathfrak{O}_0 \times \mathfrak{O}_1 \times \ldots \times \mathfrak{O}_n \mapsto A_i$
  - Cheap to compute, used during deployment

Large action space makes multiagent policies expensive to train

Summary

Summary

- Explained why RL research focuses on video games
  - MDPs are not a good representation of real tasks
- Introduced POMDPs that can model almost anything
  - Difference between observation and state
  - Concatenation of all observations and actions is a Markov state
    - Can be inefficient
    - Use RNN or attention to produce latent Markov state
- Multiagent RL also uses POMDPs!
  - Use graph neural network or attention